

TP01_Pandas_Tutorial_Sample

April 30, 2025

Copyright 2017 Google LLC.

```
[ ]: # Licensed under the Apache License, Version 2.0 (the "License");  
# you may not use this file except in compliance with the License.  
# You may obtain a copy of the License at  
#  
# https://www.apache.org/licenses/LICENSE-2.0  
#  
# Unless required by applicable law or agreed to in writing, software  
# distributed under the License is distributed on an "AS IS" BASIS,  
# WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.  
# See the License for the specific language governing permissions and  
# limitations under the License.
```

1 Intro to pandas

Learning Objectives: * Gain an introduction to the **DataFrame** and **Series** data structures of the *pandas* library * Access and manipulate data within a **DataFrame** and **Series** * Import CSV data into a *pandas DataFrame* * Reindex a **DataFrame** to shuffle data

pandas is a column-oriented data analysis API. It's a great tool for handling and analyzing input data, and many ML frameworks support *pandas* data structures as inputs. Although a comprehensive introduction to the *pandas* API would span many pages, the core concepts are fairly straightforward, and we'll present them below. For a more complete reference, the [pandas docs site](#) contains extensive documentation and many tutorials.

1.1 Basic Concepts

The following line imports the *pandas* API and prints the API version:

```
[ ]: from __future__ import print_function  
  
import pandas as pd  
pd.__version__
```

The primary data structures in *pandas* are implemented as two classes:

- **DataFrame**, which you can imagine as a relational data table, with rows and named columns.
- **Series**, which is a single column. A **DataFrame** contains one or more **Series** and a name for each **Series**.

The data frame is a commonly used abstraction for data manipulation. Similar implementations exist in [Spark](#) and [R](#).

One way to create a **Series** is to construct a **Series** object. For example:

```
[ ]: pd.Series(['San Francisco', 'San Jose', 'Sacramento'])
```

DataFrame objects can be created by passing a dict mapping string column names to their respective **Series**. If the **Series** don't match in length, missing values are filled with special **NA/NaN** values. Example:

```
[ ]: city_names = pd.Series(['San Francisco', 'San Jose', 'Sacramento'])
     population = pd.Series([852469, 1015785, 485199])

     pd.DataFrame({ 'City name': city_names, 'Population': population })
```

But most of the time, you load an entire file into a **DataFrame**. The following example loads a file with California housing data. Run the following cell to load the data and create feature definitions:

```
[ ]: california_housing_dataframe = pd.read_csv("https://download.mlcc.google.com/
     ↪mledu-datasets/california_housing_train.csv", sep=",")
     california_housing_dataframe.describe()
```

The example above used **DataFrame.describe** to show interesting statistics about a **DataFrame**. Another useful function is **DataFrame.head**, which displays the first few records of a **DataFrame**:

```
[ ]: california_housing_dataframe.head()
```

Another powerful feature of *pandas* is graphing. For example, **DataFrame.hist** lets you quickly study the distribution of values in a column:

```
[ ]: california_housing_dataframe.hist('housing_median_age')
```

1.2 Accessing Data

You can access **DataFrame** data using familiar Python dict/list operations:

```
[ ]: cities = pd.DataFrame({ 'City name': city_names, 'Population': population })
     print(type(cities['City name']))
     cities['City name']
```

```
[ ]: print(type(cities['City name'][1]))
     cities['City name'][1]
```

```
[ ]: print(type(cities[0:2]))
     cities[0:2]
```

In addition, *pandas* provides an extremely rich API for advanced [indexing and selection](#) that is too extensive to be covered here.

1.3 Manipulating Data

You may apply Python's basic arithmetic operations to **Series**. For example:

```
[ ]: population / 1000.
```

NumPy is a popular toolkit for scientific computing. *pandas* **Series** can be used as arguments to most NumPy functions:

```
[ ]: import numpy as np

     np.log(population)
```

For more complex single-column transformations, you can use **Series.apply**. Like the Python **map function**, **Series.apply** accepts as an argument a **lambda function**, which is applied to each value.

The example below creates a new **Series** that indicates whether **population** is over one million:

```
[ ]: population.apply(lambda val: val > 1000000)
```

Modifying **DataFrames** is also straightforward. For example, the following code adds two **Series** to an existing **DataFrame**:

```
[ ]: cities['Area square miles'] = pd.Series([46.87, 176.53, 97.92])
     cities['Population density'] = cities['Population'] / cities['Area square_
     ↪miles']
     cities
```

1.4 Exercise #1

Modify the **cities** table by adding a new boolean column that is True if and only if *both* of the following are True:

- The city is named after a saint.
- The city has an area greater than 50 square miles.

Note: Boolean **Series** are combined using the bitwise, rather than the traditional boolean, operators. For example, when performing *logical and*, use **&** instead of **and**.

Hint: “San” in Spanish means “saint.”

```
[ ]: # Your code here
```

1.4.1 Solution

Click below for a solution.

```
[ ]: cities['Is wide and has saint name'] = (cities['Area square miles'] > 50) &_
     ↪cities['City name'].apply(lambda name: name.startswith('San'))
     cities
```

1.5 Indexes

Both `Series` and `DataFrame` objects also define an `index` property that assigns an identifier value to each `Series` item or `DataFrame` row.

By default, at construction, *pandas* assigns index values that reflect the ordering of the source data. Once created, the index values are stable; that is, they do not change when data is reordered.

```
[ ]: city_names.index
```

```
[ ]: cities.index
```

Call `DataFrame.reindex` to manually reorder the rows. For example, the following has the same effect as sorting by city name:

```
[ ]: cities.reindex([2, 0, 1])
```

Reindexing is a great way to shuffle (randomize) a `DataFrame`. In the example below, we take the index, which is array-like, and pass it to NumPy's `random.permutation` function, which shuffles its values in place. Calling `reindex` with this shuffled array causes the `DataFrame` rows to be shuffled in the same way. Try running the following cell multiple times!

```
[ ]: cities.reindex(np.random.permutation(cities.index))
```

For more information, see the [Index documentation](#).

1.6 Exercise #2

The `reindex` method allows index values that are not in the original `DataFrame`'s index values. Try it and see what happens if you use such values! Why do you think this is allowed?

```
[ ]: # Your code here
```

1.6.1 Solution

Click below for the solution.

If your `reindex` input array includes values not in the original `DataFrame` index values, `reindex` will add new rows for these “missing” indices and populate all corresponding columns with NaN values:

```
[ ]: cities.reindex([0, 4, 5, 2])
```

This behavior is desirable because indexes are often strings pulled from the actual data (see the [pandas reindex documentation](#) for an example in which the index values are browser names).

In this case, allowing “missing” indices makes it easy to reindex using an external list, as you don't have to worry about sanitizing the input.